

Algorithmique

Récurtivité

Cédric Lhoussaine

Université de Lille / IUT de Lille

Principes de récursivité

Définition Un fonction **récursive** est une fonction qui s'appelle elle-même.

Définition Une fonction **récursive** est une fonction qui s'appelle elle-même.

Permet de ramener la résolution d'un problème compliqué à la résolution d'un (ou plusieurs) problèmes plus simples

Exemple (Factorielle - maths)

$$n! = 1 * 2 * \dots * n$$

ou bien...

$$0! = 1 \quad \text{cas de base}$$

$$n! = n * (n - 1)! \text{ si } n \geq 1 \text{ cas récursif}$$

Principes de récursivité

Définition Une fonction **récursive** est une fonction qui s'appelle elle-même.

Permet de ramener la résolution d'un problème compliqué à la résolution d'un (ou plusieurs) problèmes plus simples

Exemple (Factorielle - maths)

$$n! = 1 * 2 * \dots * n$$

ou bien...

$$0! = 1 \quad \text{cas de base}$$

$$n! = n * (n - 1)! \text{ si } n \geq 1 \text{ cas récursif}$$

Exemple (Factorielle - algo)

```
1: function FACTO(n)  
2:   if n = 0 then  
3:     return 1  
4:   else  
5:     return n * Facto(n - 1)  
6:   end  
7: end
```

Principes de récursivité: les poupées russes

on doit garantir la terminaison:

- Il doit toujours y avoir (au moins) un cas de terminaison (i.e. sans appel récursif)
- Chaque appel récursif doit se « rapproche » (strictement) un peu plus de la terminaison



```
1: function FACTO( $n$ )  
2:   if  $n = 0$  then  
3:     return 1           ▷ cas terminal: pas d'appel récursif  
4:   else  
5:     return  $n * \text{Facto}(n - 1)$   ▷ cas récursif sur  $n - 1 < n$   
6:   end  
7: end
```

Méthode du « **diviser pour régner** »:

- **Diviser** le problème en sous-problèmes

Méthode du « **diviser pour régner** »:

- **Diviser** le problème en sous-problèmes
- **Résoudre** chacun des sous-problèmes de façon récursive sauf ceux d'une taille suffisamment petite pour être résolus directement

Méthode du « **diviser pour régner** »:

- **Diviser** le problème en sous-problèmes
- **Résoudre** chacun des sous-problèmes de façon récursive sauf ceux d'une taille suffisamment petite pour être résolus directement
- **Combiner** les solutions aux sous-problèmes pour obtenir une solution au problème d'origine

Récurtivité : pour ou contre ...?

Avantages:

- Concision
- Clarté / compréhension
- Calcul de coût
- (preuve de correction)

¹mais, dans certains cas (récurtivité terminale), le compilateur (mais pas celui de java !) peut optimiser les programmes récursifs en les transformant automatiquement en programmes itératifs

Récurtivité : pour ou contre ...?

Avantages:

- Concision
- Clarté / compréhension
- Calcul de coût
- (preuve de correction)

Inconvénients:

- nécessite plus d'espace mémoire (croissance de la pile d'exécution¹)
- parfois moins efficace que des algorithmes itératifs

¹mais, dans certains cas (récurtivité terminale), le compilateur (mais pas celui de java !) peut optimiser les programmes récursifs en les transformant automatiquement en programmes itératifs

Comment calculer le coût de cet algorithme récursif ?

(au tableau...)

Algorithm 1. – Fibonacci

```
1: function FACTO( $n$ )  
2:   if  $n = 0$  then  
3:     return 1  
4:   else  
5:     return  $n * \text{Facto}(n - 1)$   
6:   end  
7: end
```

Exemple: coût de l'algorithme FACTO

Équation de récurrence du coût:

$$T(0) = c_1 + c_2$$

$$T(n) = (c_1 + c_3) + T(n-1) \quad \text{pour } n \geq 1$$

Solution de la forme:

$$T(n) = cn + c'$$

Quelle est la complexité asymptotique de cet algorithme (Wooclap)

Exemple 2: Fibonacci

Définition (Fibonacci - maths)

$$\text{Fibo}(0) = 0$$

$$\text{Fibo}(1) = 1$$

$$\text{Fibo}(n) = \text{Fibo}(n - 1) + \text{Fibo}(n - 2)$$

Exemple 2: Fibonacci

Définition (Fibonacci - maths)

$$\text{Fibo}(0) = 0$$

$$\text{Fibo}(1) = 1$$

$$\text{Fibo}(n) = \text{Fibo}(n - 1) + \text{Fibo}(n - 2)$$

Algorithm 3. – Fibonacci

```
1: function FIBO(n)
2:   if n ≤ 1 then
3:     return n
4:   else
5:     return Fibo(n − 1) + Fibo(n − 2)
6:   end
7: end
```

- Fibo(4) ? → arbres des appels récurifs (*au tableau ...*)

Établissez l'équation de récurrence du coût de cet algorithme

(au tableau...)

Calcul du coût de l'algorithme (naïf) de Fibonacci

$$T(n) = O(2^n) \text{ et } T(n) = \Omega(2^{\frac{n}{2}}) \Rightarrow \text{algorithme exponentiel}$$

Précisément: $T(n) = \Theta(\varphi^n)$ où φ est le nombre d'or (~ 1.618)